

CSE114 – Labwork 9.2

Labwork 9.2 – Recursive Operations

General Information and Submission Rules

By the end of the labwork, you must submit it to Yulearn as:

```
studentID_fullname_labwork9.c
```

Important:

- Incorrect naming or file format will result in a loss of 5 points from your total grade.
- Only use concepts covered in the labs so far.
- **All functions in this labwork must be implemented using recursion (no loops allowed in recursive functions).**

Question 1 – Display Message (10p) EASY

Create a new function `welcome` that displays a welcome message and a list of all available operations to the user.

The function will expect a decision from the user and will execute related functions until the user exits.

Print each line separately.

```
Welcome to Labwork 9 Recursive Operations! By using this machine you can
do:
1. Fibonacci Number
2. Array Sum
3. Array Maximum
4. Array Minimum
5. Check Palindrome (String)
6. String Length
7. Count Character
8. Print Array Forward
9. Print Array Backward
10. Exit
Make your choice:
```

Question 2 – Fibonacci Number (10p) EASY

Create a recursive function `fibonacci` that takes:

- an integer `n` representing the position in the Fibonacci sequence

The function should return the nth Fibonacci number using recursion.

Mathematical Definition:

```
Fib(0) = 1
Fib(1) = 1
Fib(n) = Fib(n-2) + Fib(n-1) for n >= 2
```

The sequence: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

Example:

```
>> Enter position: 6
Fibonacci sequence up to position 6:
Fib(0) = 1
Fib(1) = 1
Fib(2) = 2
Fib(3) = 3
Fib(4) = 5
Fib(5) = 8
Fib(6) = 13
```

Question 3 – Array Sum (10p) EASY

Create a recursive function `array_sum` that takes:

- a pointer to the first element of an integer array
- the size of the array

The function should calculate and return the sum of all elements in the array using recursion.

Define the following array in main:

```
int numbers[] = {5, 10, 15, 20, 25};
```

Example:

```
Array: 5 10 15 20 25
Total sum: 75
```

Question 4 – Array Maximum (10p) EASY

Create a recursive function `array_max` that takes:

- a pointer to the first element of an integer array
- the size of the array

The function should find and return the maximum element in the array using recursion.

Define the following array in main:

```
int numbers[] = {23, 45, 12, 67, 34, 89, 21};
```

Example:

```
Array: 23 45 12 67 34 89 21  
Maximum element: 89
```

Question 5 – Array Minimum (10p) EASY

Create a recursive function `array_min` that takes:

- a pointer to the first element of an integer array
- the size of the array

The function should find and return the minimum element in the array using recursion.

Example:

```
Array: 23 45 12 67 34 89 21  
Minimum element: 12
```

Question 6 – String Length (10p) EASY

Create a recursive function `str_length` that takes:

- a pointer to the first character of a string

The function should calculate and return the length of the string using recursion (without using `strlen`).

Example:

```
>> Enter a string: hello  
Length: 5
```

Another Example:

```
>> Enter a string: recursion
Length: 9
```

Question 7 – Check Palindrome (15p) 🟡 MEDIUM

Create a recursive function `is_palindrome` that takes:

- a pointer to the first character of a string
- a pointer to the last character of the string

The function should check if the given string is a palindrome (reads the same forwards and backwards) using recursion.

Example 1:

```
>> Enter a string: radar
Comparing: 'r' and 'r' – Match!
Comparing: 'a' and 'a' – Match!
Comparing: 'd' and 'd' – Match!
"radar" is a palindrome!
```

Example 2:

```
>> Enter a string: hello
Comparing: 'h' and 'o' – No match!
"hello" is not a palindrome.
```

Question 8 – Count Character in String (15p) 🟡 MEDIUM

Create a recursive function `count_char` that takes:

- a pointer to a string
- a character to search for

The function should count how many times the given character appears in the string using recursion.

Example:

```
>> Enter a string: mississippi
>> Enter character to count: s
Character 's' appears 4 times.
```

Another Example:

```
>> Enter a string: programming
>> Enter character to count: m
Character 'm' appears 2 times.
```

Question 9 – Print Array Forward (15p) 🟡 MEDIUM

Create a recursive function `print_forward` that takes:

- a pointer to the first element of an integer array
- the size of the array

The function should print all elements of the array from first to last using recursion.

Example:

```
Array: 10 20 30 40 50
Forward: 10 20 30 40 50
```

Question 10 – Print Array Backward (15p) 🟡 MEDIUM

Create a recursive function `print_backward` that takes:

- a pointer to the first element of an integer array
- the size of the array

The function should print all elements of the array from last to first using recursion.

Example:

```
Array: 10 20 30 40 50
Backward: 50 40 30 20 10
```

Question 11 – Search in Array (20p) 🔴 HARD

Create a recursive function `search_array` that takes:

- a pointer to the first element of an integer array
- the size of the array
- the element to search for

The function should return the **index** of the first occurrence of the element, or `-1` if not found.

Example:

```
Array: 10 25 30 45 50 25 60
>> Enter element to search: 25
Element 25 found at index 1
```

Another Example:

```
>> Enter element to search: 100
Element 100 not found.
```

Question 12 – Reverse String (20p) HARD

Create a recursive function `reverse_string` that takes:

- a pointer to a string
- starting index
- ending index

The function should reverse the string **in place** using recursion.

Hint: Swap first and last characters, then recursively reverse the middle part.

Example:

```
>> Enter a string: hello
Original: hello
Reversed: olleh
```

Another Example:

```
>> Enter a string: recursion
Original: recursion
Reversed: noisrucer
```

BONUS Question – Tower of Hanoi (20p) HARD

Create a recursive function `hanoi` that takes:

- number of disks `n`
- source peg name (char)
- auxiliary peg name (char)

- destination peg name (char)

The function should print all moves required to solve the Tower of Hanoi puzzle.

Rules:

1. Only one disk can be moved at a time
2. A larger disk cannot be placed on a smaller disk
3. Move all disks from source to destination using auxiliary peg

Example:

```
>> Enter number of disks: 3
Move disk from A to C
Move disk from A to B
Move disk from C to B
Move disk from A to C
Move disk from B to A
Move disk from B to C
Move disk from A to C
Total moves: 7
```

BONUS Question – Check Sorted Array (15p) HARD

Create a recursive function `is_sorted` that takes:

- a pointer to the first element of an integer array
- the size of the array

The function should check if the array is sorted in ascending order using recursion.

Example 1:

```
Array: 10 20 30 40 50
Array is sorted: YES
```

Example 2:

```
Array: 10 30 20 40 50
Array is sorted: NO
```

BONUS Question – Sum of Array Elements at Even Indices (15p) MEDIUM

Create a recursive function `sum_even_indices` that takes:

- a pointer to the first element of an integer array
- the size of the array

The function should sum only the elements at even indices (0, 2, 4, ...).

Example:

```
Array: 10 20 30 40 50 60
Indices: 0 1 2 3 4 5
Sum of elements at even indices (10 + 30 + 50): 90
```

Grading Distribution:

Difficulty	Questions	Points Each	Total
 Easy	Q1-Q6	10p	60p
 Medium	Q7-Q10	15p	60p
 Hard	Q11-Q12	20p	40p
 Bonus	Hanoi, Sorted, Even Indices	15-20p	50p (Extra)

Base Total: 160 points (50 bonus points available)

Hints

1. Array Recursion Pattern:

```
int array_func(int *arr, int size) {
    // Base case
    if (size == 1) return arr[0];

    // Recursive case - two approaches:
    // Approach 1: Process first element
    return arr[0] + array_func(arr + 1, size - 1);

    // Approach 2: Process last element
    return arr[size-1] + array_func(arr, size - 1);
}
```

2. String Recursion Pattern:

```
int string_func(char *str) {
    // Base case - empty string or null terminator
    if (*str == '\0') return 0;
```

```

// Recursive case
return 1 + string_func(str + 1);
}

```

3. Two-Pointer String Pattern (for palindrome, reverse):

```

int check_string(char *first, char *last) {
    // Base case – pointers have met or crossed
    if (first >= last) return 1;

    // Check and recurse
    if (*first != *last) return 0;
    return check_string(first + 1, last - 1);
}

```

4. Finding Max/Min in Array:

```

int array_max(int *arr, int size) {
    if (size == 1) return arr[0];

    int max_rest = array_max(arr + 1, size - 1);
    return (arr[0] > max_rest) ? arr[0] : max_rest;
}

```

5. Tower of Hanoi Logic:

```

void hanoi(int n, char from, char aux, char to) {
    if (n == 1) {
        printf("Move disk from %c to %c\n", from, to);
        return;
    }
    hanoi(n-1, from, to, aux);    // Move n-1 disks to auxiliary
    printf("Move disk from %c to %c\n", from, to); // Move largest disk
    hanoi(n-1, aux, from, to);    // Move n-1 disks to destination
}

```